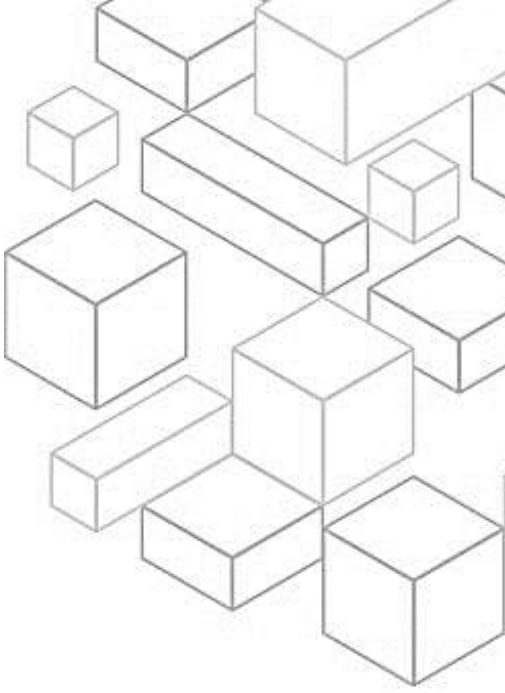
Transcript for Module 6 AWS Well-Architected: Deep Dive on the Reliability Pillar



1.1 Welcome!

Welcome to module six of AWS Well-Architected: Deep Dive on the Reliability Pillar.

1.2 Learning objectives

In this module, you will learn about the reliability pillar of the AWS Well-Architected Framework. You will also learn the design principles and best practices of the reliability pillar.

1.3 Reliability Pillar Overview

First, you will get an overview of the reliability pillar.

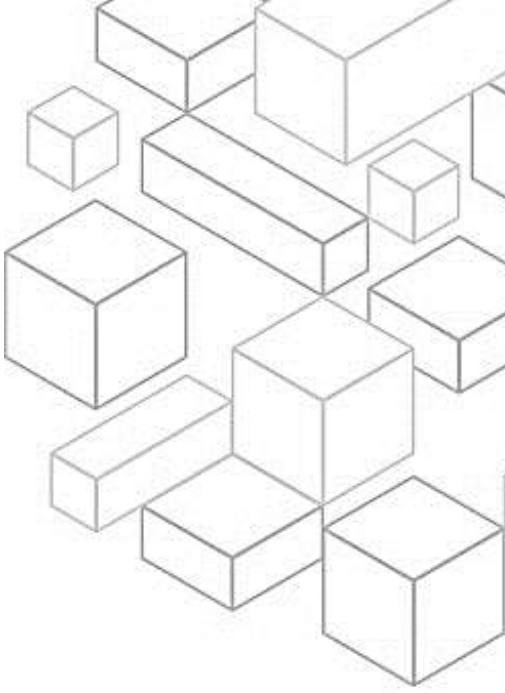
1.4 Pillars of AWS Well-Architected

There are currently six pillars in the Well-Architected Framework: operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability. These pillars are the foundations of the architecture for your technology solutions in the cloud.

This module will focus on the reliability pillar.

1.5 What is the reliability pillar?

What is the reliability pillar? And how should we think about it? Reliability is the ability of a workload to perform its required function correctly and consistently over an expected period of time. The reliability pillar focuses on the ability of a workload to perform its intended function correctly and consistently when expected to. This includes the ability to operate and test a workload through its total lifecycle.



1.6 Reliability Design Principles

In the cloud, a number of principles can help you increase reliability. You will now learn more about these design principles.

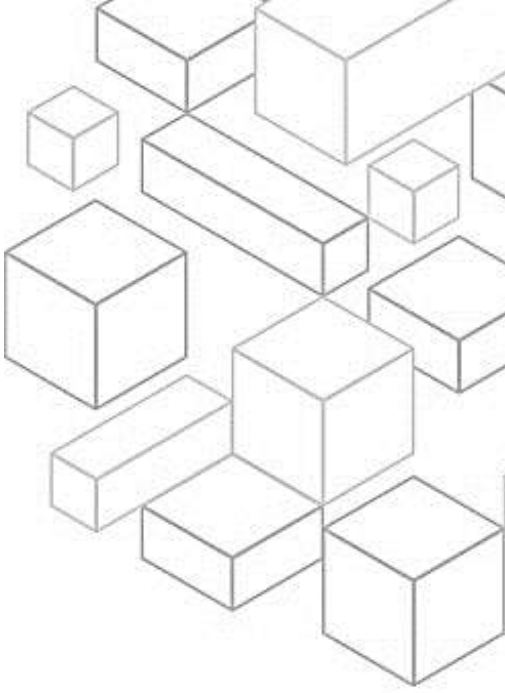
1.7 Reliability design principles

First, review the reliability design principles before diving into best practices. The design principles help shape a mental model for the reliability pillar, especially if you are coming from a traditional on-premises environment. One principle is automatically recover from failure. You do so by monitoring a workload for key performance indicators (KPIs), and then start an automation to perform a specific job when threshold is breached. These KPIs should be a measure of business value, not of the technical aspects of the operation of the service. You can have automatic notification and tracking of failures and automated recovery processes that work around or repair the failure. With more sophisticated automation, it's possible to anticipate and remediate failures before they occur.

The next principle is test recovery procedures. In an on-premises environment, testing is often conducted to prove that the workload works in a particular scenario. Testing is not typically used to validate recovery strategies. In the cloud, you can test how your workload fails, and you can validate your recovery procedures. You can use automation to simulate different failures or to recreate scenarios that led to failures before. This approach exposes failure pathways that you can test and fix before a real failure scenario occurs, thus reducing risk.

Scale horizontally to increase aggregate workload availability. Replace one large resource with multiple small resources to reduce the impact of a single failure on the overall workload. Distribute requests across multiple, smaller resources to ensure that they don't share a common point of failure.

Next, stop guessing capacity. A common cause of failure in on-premises workloads is resource saturation, when the demands placed on a workload exceed the capacity of that workload. One example is denial of service attacks. In the cloud, you can monitor demand and workload utilization, and automate the addition or removal of resources to maintain the optimal level to satisfy demand without over-provisioning or under-provisioning. There are still limits, but some quotas can be controlled and others can be managed.



Another principle is how to manage change through automation. Changes to your infrastructure should be made using automation. The changes that need to be managed include changes to the automation, which then can be tracked and reviewed.

1.8 Reliability Best Practices

In the cloud, best practices can help you increase reliability. In this section, you will learn more about these best practices.

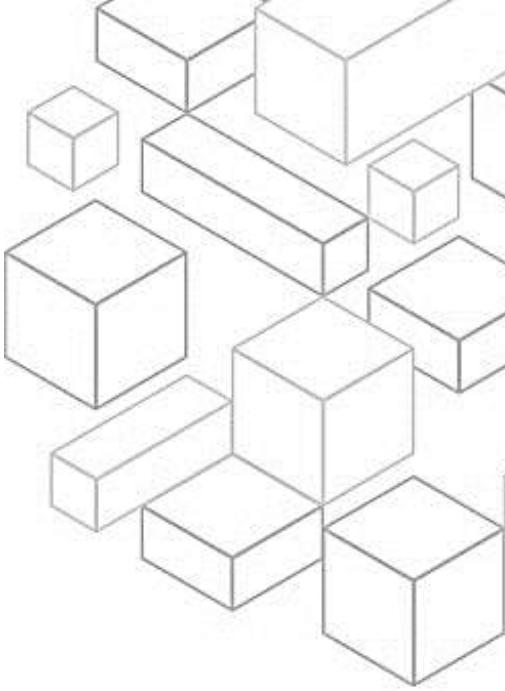
1.9 Reliability best practice areas

The best practices in the reliability pillar are organized into four areas, with the first being foundations. The scope of foundational requirements extends beyond a single workload or project. Before architecting any system, foundational requirements that influence reliability should be in place. For example, you must have sufficient network bandwidth to your data center. In an on-premises environment, these requirements can cause long lead times due to dependencies and therefore must be incorporated during initial planning. With AWS, however, most of these foundational requirements are already incorporated or can be addressed as needed. The cloud is designed to be nearly limitless, so it's the responsibility of AWS to satisfy the requirement for sufficient networking and compute capacity. This leaves you free to change resource size and allocations on demand.

The next best practice is workload architecture. A reliable workload starts with upfront design decisions for both software and infrastructure. Your architecture choices will impact your workload behavior across all six AWS Well-Architected pillars. For reliability, there are specific patterns you must follow.

For the change management best practice, changes to your workload or its environment must be anticipated and accommodated for reliable workload operation. Changes include those imposed on your workload, such as spikes in demand. They also include internal changes, such as feature deployments and security patches.

Finally, for failure management, low-level hardware component failures are something to be dealt with every day in an on-premises data center. In the



cloud, however, you should be protected against most of these types of failures. For example, Amazon Elastic Block Store, or Amazon EBS, volumes are placed in a specific Availability Zone where they are automatically replicated to protect you from the failure of a single component. All EBS volumes are designed for five nines of availability. Amazon Simple Storage Service, or Amazon S3, objects are stored across a minimum of three Availability Zones, providing eleven nines of durability for objects over a given year. Regardless of your cloud provider, there is the potential for failures to impact your workload. Therefore, you must take steps to implement resiliency if you need your workload to be reliable. A prerequisite to applying the best practices discussed here is that you must ensure that the people who are designing, implementing, and operating your workloads are trained for the business objectives and reliability goals to achieve them.

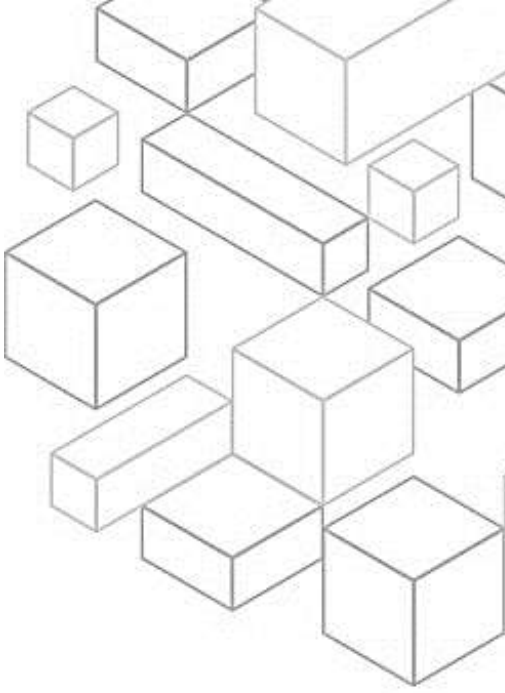
1.10 Foundations

In the foundations best practice area, scope can extend beyond a single workload or project. It needs to be understood and in place before architecting any system. If you don't, you might face long lead times and blockers along the way. Get these taken care of early so you can be free to change resource size and allocation on demand. In this section, you will learn how to manage service quotas or constraints and plan your network topology.

1.11 Manage service quotas and constraints

Manage service quotas and constraints. For cloud-based workload architectures, there are service quotas, which are also referred to as service limits. These quotas exist to prevent accidentally provisioning more resources than you need. They also help limit request rates on API operations to protect services from abuse. There are also resource constraints, for example, the rate that you can push bits down a fiber-optic cable, or the amount of storage on a physical disk.

It is a good idea to be aware of your default quotas and quota increase requests for your workload architecture. You additionally should know which resource constraints, such as disk or network, are potentially impactful. You can also manage service quotas across accounts and Regions. If you are using multiple AWS accounts or AWS Regions, ensure that you request the appropriate quotas in all environments in which your production workloads



run. Service quotas are tracked per account. Unless otherwise noted, each quota is AWS Region-specific. In addition to the production environments, also manage quotas in all applicable nonproduction environments so that testing and development are not hindered.

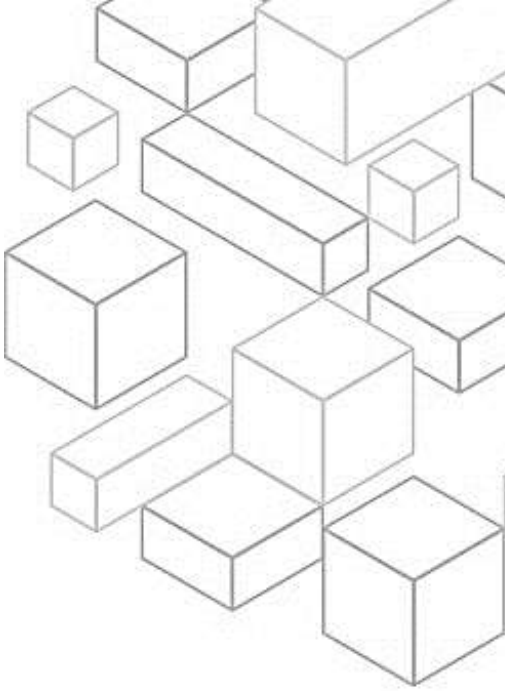
Next, accommodate fixed service quotas and constraints through architecture. Be aware of unchangeable service quotas and physical resources, and architect to prevent these from impacting reliability. You can also monitor and manage quotas. Evaluate your potential usage, and then increase your quotas appropriately with room for planned growth in usage. Automate quota management by implementing tools to alert you when a threshold approaches. You can automate quota increase requests using Service Quotas APIs.

Finally, ensure that a sufficient gap exists between the current quotas and the maximum usage to accommodate failover. When a resource fails, it might still be counted against quotas until it is successfully terminated. Verify that your quotas cover the overlap of all failed resources with replacements before the failed resources are terminated. Consider an Availability Zone failure when calculating this gap.

1.12 Plan your network topology

Plan your network topology. Workloads often exist in multiple environments. These include multiple cloud environments (both publicly accessible and private) and possibly your existing data center infrastructure. Plans must include network considerations, such as intrasystem and intersystem connectivity, public IP address management, private IP address management, and domain name resolution. When architecting systems using IP address-based networks, you must plan network topology and anticipate possible failures. It is important to accommodate future growth and integration with other systems and their networks.

Be sure to use highly available network connectivity for your workload public endpoints. These endpoints and the routing to them must be highly available. To achieve this, use highly available DNS, content delivery networks (CDNs), API Gateway, load balancing, or reverse proxies. Provision redundant connectivity between private networks in the cloud and on-premises environments. Use multiple AWS Direct Connect connections or virtual private network tunnels between separately deployed private networks. You can also



use multiple Direct Connect locations for high availability. If using multiple AWS Regions, ensure redundancy in at least two of them.

You can also ensure IP subnet allocation accounts for expansion and availability. Amazon Virtual Private Cloud, or Amazon VPC, IP address ranges must be large enough to accommodate workload requirements. This includes factoring in future expansion and allocation of IP addresses to subnets across Availability Zones. This also accounts for load balancers, EC2 instances, and container-based applications.

Consider hub-and-spoke topologies over many-to-many mesh if more than two network address spaces, such as VPCs and on-premises networks, are connected through VPC peering, Direct Connect, or VPN. AWS Transit Gateway is one example of a hub-and-spoke model.

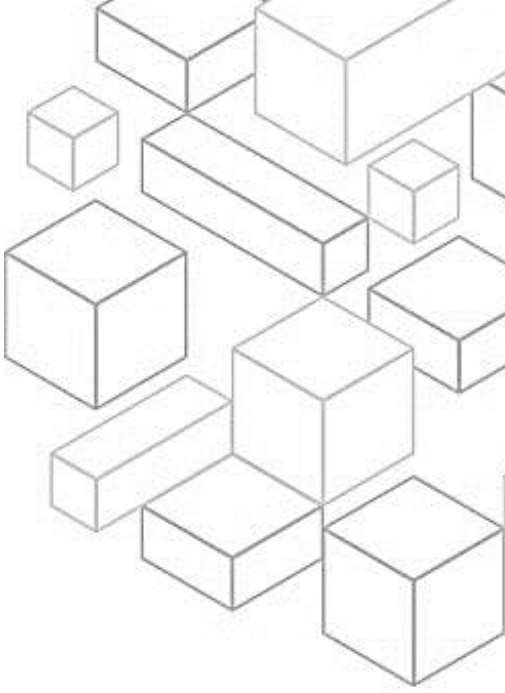
Finally, enforce non-overlapping private IP address ranges in all private address spaces where they are connected. The IP address ranges of each of your VPCs must not overlap when peered or connected through VPN. You must similarly avoid IP address conflicts between a VPC and on-premises environments or with other cloud providers that you use. You should also have a way to allocate private IP address ranges when needed.

1.13 Workload Architecture

Workload architecture is the next reliability best practice area. A reliable workload starts with upfront design decisions for both software and infrastructure. Your architecture choices will impact your workload behavior across all six AWS Well-Architected pillars. For reliability, there are specific patterns to include. The following section explains best practices to use with these patterns for reliability. You will learn about designing your workload service architecture. You will also consider interactions in a distributed system to prevent, mitigate, or withstand failures.

1.14 Design your workload service architecture

Design your workload service architecture. Build highly scalable and reliable workloads using a service-oriented architecture or a microservices architecture. Service-oriented architecture is the practice of making software components



reusable through service interfaces. Microservices architecture goes further to make components smaller and simpler.

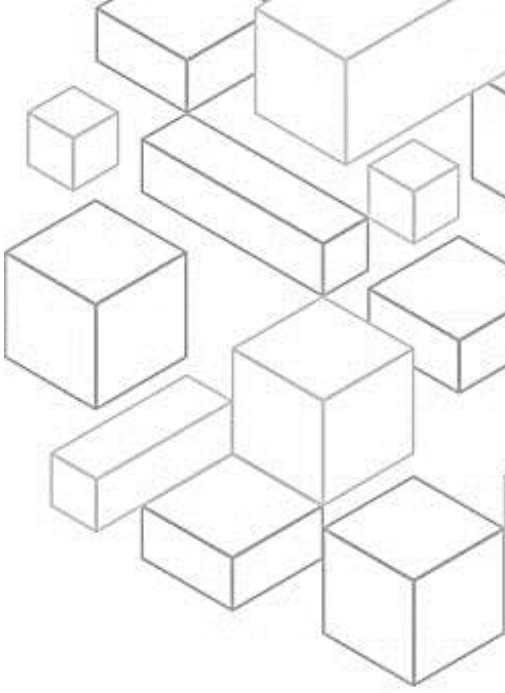
To design your architecture, first choose how to segment your workload. Workload segmentation is important when determining the resilience requirements of your application. Avoid monolithic architecture whenever possible. Instead, carefully consider which application components can be broken out into microservices. Depending on your application requirements, this might end up combining service-oriented architecture with microservices where possible. Workloads that are capable of statelessness are more capable of being deployed as microservices.

Next, build services focused on specific business domains and functionality. Service-oriented architecture builds services with well-delineated functions, defined by business needs. Microservices use domain models and bounded context to limit this further so that each service does just one thing. Focusing on functionality helps you differentiate the reliability requirements of each service and better target investments. You can also do faster organization scaling with a concise business problem and a small team associated with each service.

Finally, provide service contracts per API. Service contracts are documented agreements between teams on service integration. They include a machine-readable API definition, rate limits, and performance expectations. A versioning strategy helps your clients continue using the existing API to migrate their applications to a newer API when they are ready. Deployment can happen anytime, as long as the contract is not violated. The service provider team can use the technology stack of their choice to satisfy the API contract. Similarly, the service consumer can use their own technology.

1.15 Design interactions in a distributed system to prevent failures

Design interactions in a distributed system to prevent failures. Distributed systems rely on communications networks to interconnect components, such as servers or services. Your workload must operate reliably despite data loss or latency in these networks. Components of the distributed system should operate in a way that does not negatively impact other components or the



workload. These best practices can help prevent failures and improve mean time between failures, or MTBF.

First, identify which kind of distributed system is required. Hard real-time distributed systems require responses to be given synchronously and rapidly. Soft real-time systems have a more generous time window of minutes or more for response. Offline systems handle responses through batch or asynchronous processing. Hard real-time distributed systems have the most stringent reliability requirements.

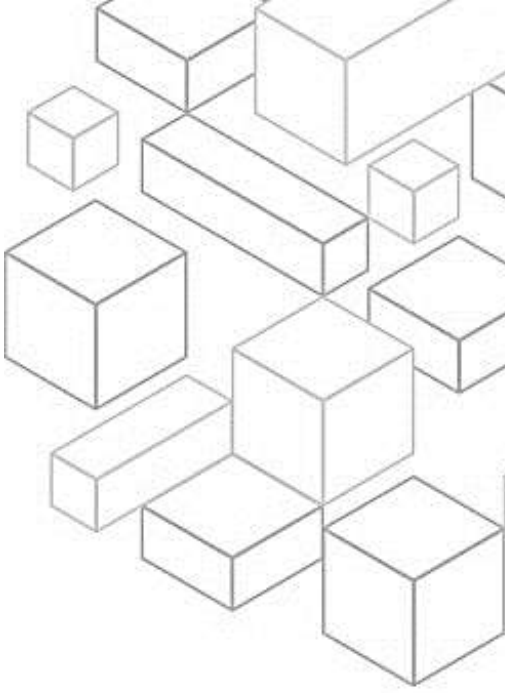
Second, implement loosely coupled dependencies. Dependencies such as queuing systems, streaming systems, workflows, and load balancers are loosely coupled. Loose coupling helps isolate behavior of a component from other components that depend on it, increasing resiliency and agility.

Third, do constant work. Systems can fail when there are large, rapid changes in load. For example, if your workload is doing a health check that monitors the health of thousands of servers, it should send the same size payload (a full snapshot of the current state) each time. Whether no servers are failing, or all of them, the health check system is doing constant work with no large, rapid changes.

Finally, make all responses idempotent. An idempotent service promises that each request is completed exactly once, such that making multiple identical requests has the same effect as making a single request. An idempotent service helps a client implement retries without fear that a request will be erroneously processed multiple times. To do this, clients can issue API requests with an idempotency token-the same token is used whenever the request is repeated. An idempotent service API uses the token to return a response identical to the response that was returned the first time that the request was completed.

1.16 Design interactions in a distributed system to mitigate

Design interactions in a distributed system to mitigate or withstand failures. Distributed systems rely on communications networks to interconnect components, such as servers or services. Your workload must operate reliably despite data loss or latency over these networks. Components of the distributed system must operate in a way that does not negatively impact other components or the workload. These best practices help workloads withstand stresses or failures, more quickly recover from them, and mitigate



the impact of such impairments. The result is improved mean time to recovery, or MTTR.

Implement graceful degradation to transform applicable hard dependencies into soft dependencies. When a component's dependencies are unhealthy, the component itself can still function, although in a degraded manner. For example, when a dependency call fails, fail over to a predetermined static response.

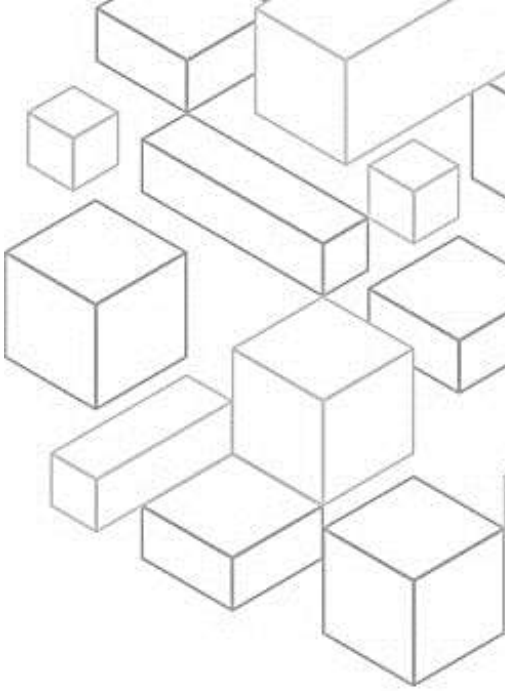
Throttling requests is a mitigation pattern to respond to an unexpected increase in demand. Some requests are honored, but those over a defined limit are rejected and return a message indicating that they have been throttled. The expectation on clients is that they will back off and abandon the request, or try again at a slower rate.

Control and limit retry calls. Use exponential backoff to retry after progressively longer intervals. Introduce jitter to randomize those retry intervals, and limit the maximum number of retries. Fail fast and limit queues: If the workload is unable to respond successfully to a request, then fail fast. This helps release resources associated with a request and permits the service to recover if it is running out of resources. If the workload is able to respond successfully, but the rate of requests is too high, you can use a queue to buffer requests instead. However, do not permit long queues that can result in serving stale requests that the client has already given up on.

Set client timeouts appropriately, verify them systematically, and do not rely on default values as they are generally set too high. This best practice applies to the client side, or sender, of the request. Make services stateless where possible. Services should either not require state, or should offload state such that between different client requests, there is no dependence on locally stored data on disk and in memory. This helps servers be replaced at will, without causing an availability impact. Finally, implement emergency levers. These rapid processes can mitigate availability impact on your workload.

1.17 Change Management

Change management is the next reliability best practice area. to your workload or its environment must be anticipated and accommodated to achieve reliable operation of the workload. Changes include those imposed on your workload such as spikes in demand. They also include those from within, such as feature



deployments and security patches. The following section explains the best practices for change management. You will learn how to monitor workload resources, design a workload to adapt to changes in demand, and implement change.

1.18 Monitor workload resources

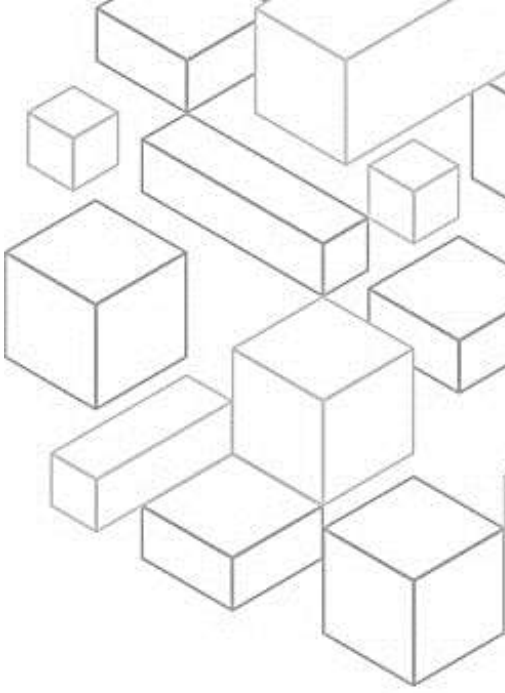
Monitor workload resources. Logs and metrics are powerful tools to gain insight into the health of your workload. You can configure your workload to monitor logs and metrics. Then, they can send notifications when thresholds are crossed or significant events occur. Monitoring helps your workload recognize when low-performance thresholds are crossed or failures occur, so it can recover automatically in response. Monitor all components for the workload. This means you can monitor the components of the workload using Amazon CloudWatch or third-party tools. Monitor AWS services with AWS Health Dashboard.

Define and calculate metrics. Store log data and apply filters where necessary to calculate metrics, such as counts of a specific log event or latency calculated from log event timestamps. Send notifications. Organizations that need to know can receive notifications when significant events occur.

Automate responses by implementing real-time processing and alarming. Use automation to take action when an event is detected, for example, to replace failed components. Perform analytics by collecting log files and metrics histories, and then analyzing these for broader trends and workload insights.

Conduct reviews regularly. Frequently review how workload monitoring is implemented and update it based on significant events and changes. Effective monitoring is driven by key business metrics. Ensure that these metrics are accommodated in your workload as business priorities change.

Finally, monitor end-to-end tracing of requests through your system. Use AWS X-Ray or third-party tools so that developers can quickly analyze and debug distributed systems. This helps them understand how their applications and underlying services are performing.



1.19 Design a workload to adapt to changes in demand

Design a workload to adapt to changes in demand. A scalable workload provides elasticity to add or remove resources automatically so that they closely match the current demand at any given point in time. Use automation when obtaining or scaling resources. When replacing impaired resources or scaling your workload, automate the process by using managed AWS services, such as Amazon S3 and AWS Auto Scaling. You can also use third-party tools and AWS SDKs to automate scaling.

Obtain resources upon detection of impairment to a workload. Scale resources reactively when necessary, if availability is impacted, to restore workload availability. You first must configure health checks and the criteria on these checks to indicate when availability is impacted by lack of resources. Then, either notify the appropriate personnel to manually scale the resource, or activate automation to automatically scale it.

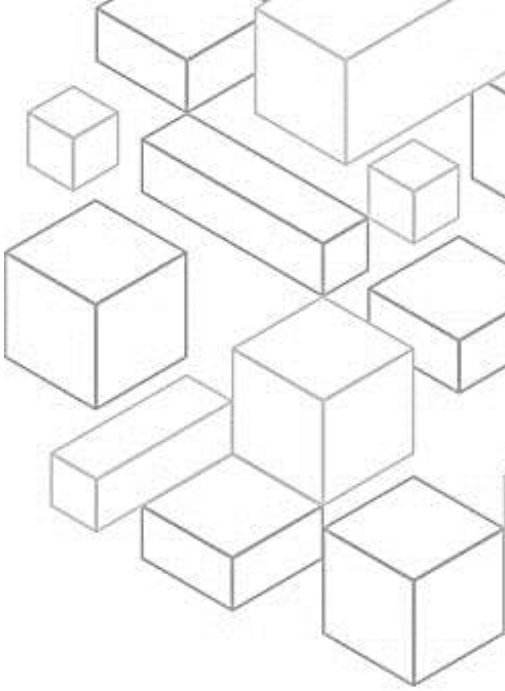
Obtain resources upon detection that more resources are needed for a workload. Scale resources proactively to meet demand and avoid availability impact. Adopt a load-testing methodology to measure if scaling activity meets workload requirements.

1.20 Implement change

Controlled changes are necessary to deploy new functionality and help ensure that the workloads and operating environment are running known properly-patched software. If these changes are uncontrolled, then it makes it difficult to predict the effect of these changes or address issues that arise because of them.

Use runbooks for standard activities such as deployment. Runbooks are the predefined procedures to achieve specific outcomes. Use runbooks to perform standard activities, whether done manually or automatically. Examples include deploying a workload, patching a workload, or making DNS modifications.

Integrate functional testing as part of your deployment. Functional tests are run as part of automated deployment. If success criteria are not met, the pipeline is halted or rolled back. These tests are run in a pre-production



environment, which is staged prior to production in the pipeline. Ideally, this is done as part of a deployment pipeline.

Integrate resiliency testing as part of your deployment. Resiliency tests, using the principles of chaos engineering, are staged and run as part of the automated deployment pipeline in a pre-production environment. They should also be run in production as part of game days.

Deploy using immutable infrastructure. Immutable infrastructure is a model that mandates that no updates, security patches, or configuration changes happen in place on production workloads. When a change is needed, the architecture is built onto new infrastructure and deployed into production. Automate deployments and patching to eliminate negative impact.

1.21 Failure Management

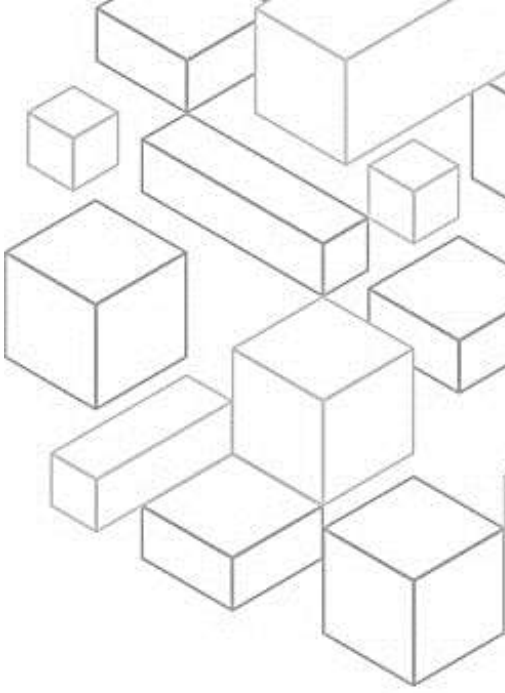
Failure Management is the next reliability best practice area. Low-level hardware component failures are something to be dealt with every day in an on-premises data center. In the cloud, however, you should be protected against most of these types of failures. Regardless of your cloud provider, there is the potential for failures to impact your workload. Therefore, you must take steps to implement resiliency if you need your workload to be reliable.

A prerequisite to applying the best practices discussed here is that you must ensure that the people designing, implementing, and operating your workloads are aware of business objectives and the reliability goals to achieve these. These people must be aware of and trained for these reliability requirements.

The following sections explain the best practices for managing failures to prevent impact on your workload.

1.22 Back up data

Back up data, applications, and configuration to meet requirements for recovery time objectives (RTO) and recovery point objectives (RPO). Identify and back up all data that needs to be backed up, or reproduce the data from sources. When selecting a backup strategy, consider the time it takes to recover data. The time needed to recover data depends on the type of backup (in the



case of a backup strategy) or the complexity of the data reproduction mechanism. This time should fall inside the RTO for the workload.

Secure and encrypt backups. Control and detect access to backups using authentication and authorization, such as AWS Identity and Access Management, or IAM. Prevent and detect whether data integrity of backups is compromised using encryption. Perform data backup automatically. Configure backups to be taken automatically based on a periodic schedule informed by the RPO or by changes in the dataset. Critical datasets with low data loss requirements need to be backed up automatically on a frequent basis, whereas less critical data where some loss is acceptable can be backed up less frequently.

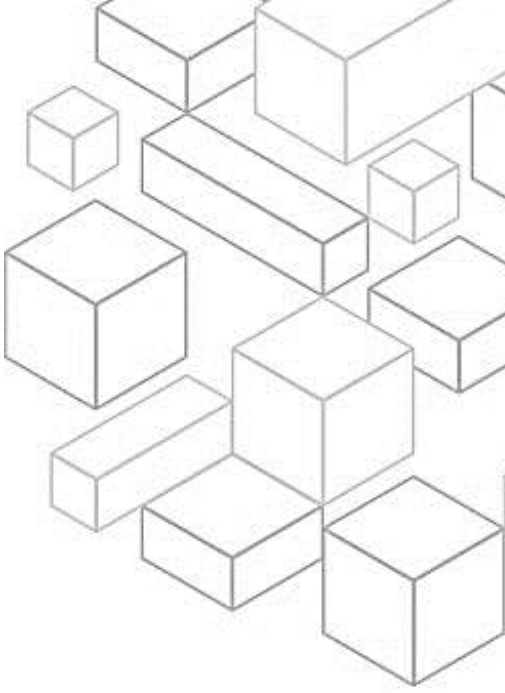
Perform periodic recovery of the data to verify backup integrity and processes. Validate that your backup process implementation meets your RTO and RPO by performing a recovery test.

1.23 Use fault isolation to protect your workload

Use fault isolation to protect your workload. Fault-isolated boundaries limit the effect of a failure inside a workload to a limited number of components. Components outside of the boundary are unaffected by the failure. Using multiple fault isolated boundaries, you can limit the impact on your workload. First, deploy the workload to multiple locations. Distribute workload data and resources across multiple Availability Zones or, where necessary, across AWS Regions. These locations can be as diverse as required.

Next, select the appropriate locations for your multi-location deployment. For high availability, always try to deploy your workload components to multiple Availability Zones. For workloads with extreme resilience requirements, carefully evaluate the options for a multi-Region architecture. You can also automate recovery for components constrained to a single location. If components of the workload can only run in a single Availability Zone or in an on-premises data center, implement the capability to do a complete rebuild of the workload inside your defined recovery objectives.

Use bulkhead architectures to limit scope of impact. Like the bulkheads on a ship, this pattern ensures that a failure is contained to a small subset of requests or clients. This helps limit the number of impaired requests so that



most can continue without error. Bulkheads for data are often called partitions, while bulkheads for services are known as cells.

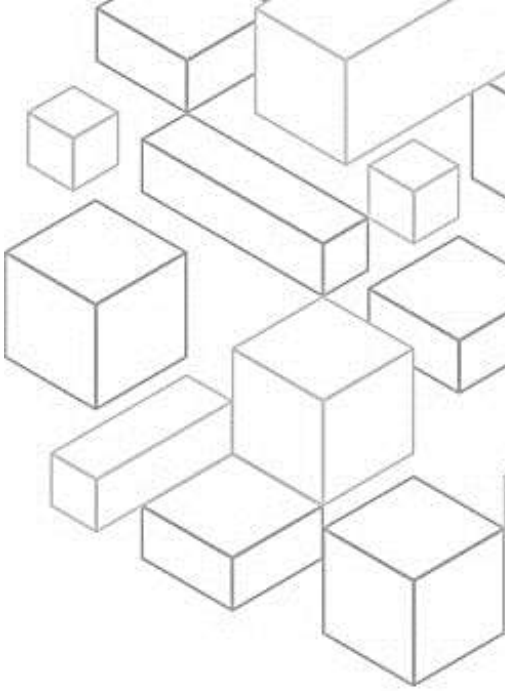
1.24 Design workload to withstand component failures

Design workload to withstand component failures. Workloads with a requirement for high availability and low mean time to recovery, or MTTR, must be architected for resiliency. Monitor all components of the workload to detect failures. Continuously monitor the health of your workload so that you and your automated systems are aware of degradation or failure as soon as they occur. Monitor for KPIs based on business value. Next, fail over to healthy resources. Ensure that, if a resource failure occurs, healthy resources can continue to serve requests. For location failures, such as Availability Zone or AWS Region, ensure that you have systems in place to fail over to healthy resources in unimpaired locations.

Automate healing on all layers. Upon detection of a failure, use automated capabilities to perform actions to remediate. Ability to restart is an important tool to remediate failures. As discussed previously for distributed systems, a best practice is to make services stateless where possible. This prevents loss of data or availability on restart.

Rely on the data plane, and not the control plane, during recovery. The control plane is used to configure resources, and the data plane delivers services. Data planes typically have higher availability design goals than control planes and are usually less complex. When implementing recovery or mitigation responses to potentially resiliency-impacting events, using control plane operations can lower the overall resiliency of your architecture.

Use static stability to prevent bimodal behavior. Bimodal behavior is when your workload exhibits different behavior under normal and failure modes. One example is relying on launching new instances if an Availability Zone fails. You should instead build workloads that are statically stable and operate in only one mode. In this case, provision enough instances in each Availability Zone to handle the workload load if one zone were removed. Then, use Elastic Load Balancing or Amazon Route 53 health checks to shift load away from the impaired instances.



Send notifications when events impact availability. Notifications are sent upon the detection of significant events, even if the issue caused by the event was automatically resolved. Finally, architect your product to meet availability targets and uptime service-level agreements, or SLAs. If you publish or privately agree to availability targets or uptime SLAs, verify that your architecture and operational processes are designed to support them.

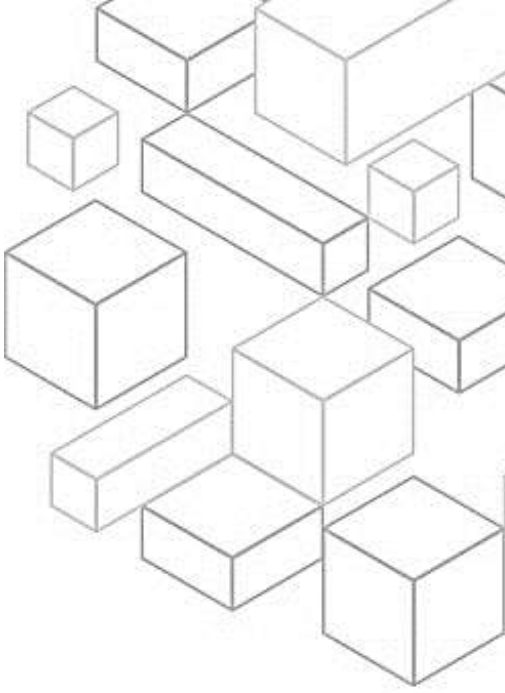
1.25 Test reliability

Test reliability. After designing your workload to be resilient to production stresses, testing is the only way to ensure that it will operate as designed to deliver the resiliency you expect. Test to validate that your workload meets functional and nonfunctional requirements because bugs or performance bottlenecks can impact its reliability. Test the resiliency of your workload to help find latent bugs that only surface in production. Exercise these tests regularly.

Use playbooks to investigate failures. Set up consistent and prompt responses to failure scenarios that are not well understood by documenting the investigation process in playbooks. Playbooks are the predefined steps performed to identify factors contributing to a failure scenario. The results from any process step are used to determine next steps until the issue is identified or escalated.

Perform post-incident analysis. Review customer-impacting events, and identify the contributing factors and preventative action items. Use this information to develop mitigations to limit or prevent recurrence. Develop procedures for prompt, effective responses. Communicate contributing factors and corrective actions as appropriate, tailored to target audiences. Document a method to communicate these causes to others as needed.

Test functional requirements using techniques such as unit tests and integration tests to validate required functionality. Also test scaling and performance requirements. Use techniques such as load testing to validate that the workload meets scaling and performance requirements. Test resiliency using chaos engineering. Run chaos experiments regularly in environments that are in or as close to production as possible to understand how your system responds to adverse conditions.



Finally, conduct game days regularly. Use game days to consistently exercise your procedures for responding to events and failures as close to production as possible. Include production environments and the people who will be involved in actual failure scenarios. Game days enforce measures to help ensure that production events do not impact users.

1.26 Plan for disaster recovery

Plan for disaster recovery. Having backups and redundant workload components in place is the start of your disaster recovery strategy. RTO and RPO are your objectives for restoring your workload. Set these based on business needs. Implement a strategy to meet these objectives, considering locations and function of workload resources and data. The probability of disruption and cost of recovery are also key factors that help inform the business value of providing disaster recovery for a workload.

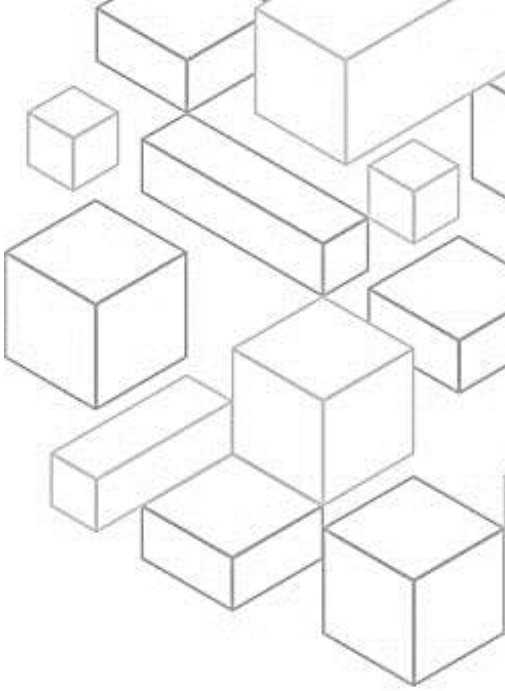
Both availability and disaster recovery rely on the same best practices. Examples include monitoring for failures, deploying to multiple locations, and performing automatic failover. However, availability focuses on components of the workload, while disaster recovery focuses on discrete copies of the entire workload. Disaster recovery has different objectives than availability, focusing on time to recovery after a disaster.

Define recovery objectives for downtime and data loss. The workload has an RTO and RPO. The RTO is the maximum acceptable delay between the interruption and restoration of service. This determines what is considered an acceptable time window when service is unavailable. The RPO is the maximum acceptable amount of time since the last data recovery point. This determines what is considered an acceptable loss of data between the last recovery point and the interruption of service.

Use defined recovery strategies to meet the recovery objectives. Define a disaster recovery strategy that meets your workload's recovery objectives. Choose a strategy such as backup and restore, active-passive, or active-active. Regularly test failover to your recovery site to ensure proper operation, and that RTO and RPO are met.

Manage configuration drift at the disaster recovery site or Region. Ensure that the infrastructure, data, and configuration are as needed at the disaster

recovery site or Region. For example, check that AMIs and service quotas are up to date. Finally, use AWS or third-party tools to automate system recovery and route traffic to the disaster recovery site or Region.



1.30 Summary

In this module, you learned the importance of the reliability pillar in the Well-Architected Framework and the value proposition for reliability in your architectures. You also learned about the design principles and best practices of the reliability pillar.

1.31 Thank you

Thank you for your participation!